

```

#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// ----- MOTOR PINS -----
const int motorPins[3] = {25, 26, 27};

// ----- POSTURE SETTINGS -----
const float postureThreshold = 2.0; // 2 degrees
float referenceAngle = 0.0;
bool calibrated = false;

// ----- CALIBRATION & FILTERING -----
float axOffset = 0;
float azOffset = 0;
float filteredAx = 0;
float filteredAz = 0;

// ----- FUNCTION DECLARATIONS -----
void calibrateMPU();
float getPostureAngle();
void activateMotors();
void deactivateMotors();

void setup() {
  Serial.begin(115200);
  delay(2000);

  Serial.println("System starting...");

  // Initialize I2C and MPU6050
  Wire.begin(21, 22);
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  Serial.println("MPU6050 connected");
  Serial.println("Stabilizing sensor...");
  delay(4000); // MPU warm-up

  // Calibrate MPU6050

```

```

Serial.println("Calibrating...");
calibrateMPU();
delay(500);

// Set posture reference
referenceAngle = getPostureAngle();
calibrated = true;

Serial.print("Reference angle: ");
Serial.println(referenceAngle);

// Motor pin setup
for (int i = 0; i < 3; i++) {
  pinMode(motorPins[i], OUTPUT);
  digitalWrite(motorPins[i], LOW);
}
}

void loop() {
  if (!calibrated) return;

  float currentAngle = getPostureAngle();
  float deviation = abs(currentAngle - referenceAngle);

  Serial.print("Angle: ");
  Serial.print(currentAngle);
  Serial.print(" Deviation: ");
  Serial.println(deviation);

  // ----- MOTOR CONTROL -----
  if (deviation > postureThreshold) {
    activateMotors(); // ALL motors ON
  } else {
    deactivateMotors(); // ALL motors OFF
  }

  delay(100);
}

// ----- CALIBRATION -----

void calibrateMPU() {
  long axSum = 0;
  long azSum = 0;
  const int samples = 3000;

```

```

for (int i = 0; i < samples; i++) {
  int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);

  axSum += ax;
  azSum += az;

  delay(2);
}

axOffset = axSum / samples;
azOffset = (azSum / samples) - 16384; // subtract gravity

Serial.print("AX offset: ");
Serial.println(axOffset);
Serial.print("AZ offset: ");
Serial.println(azOffset);
}

// ----- ANGLE COMPUTATION -----

float getPostureAngle() {
  int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);

  // Convert to g
  float axg = (ax - axOffset) / 16384.0;
  float azg = (az - azOffset) / 16384.0;

  // Low-pass filter
  filteredAx = 0.9 * filteredAx + 0.1 * axg;
  filteredAz = 0.9 * filteredAz + 0.1 * azg;

  float angle = atan2(filteredAx, filteredAz) * 180.0 / PI;
  return angle;
}

// ----- MOTOR FUNCTIONS -----

void activateMotors() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(motorPins[i], HIGH);
  }
}

```

```
void deactivateMotors() {  
  for (int i = 0; i < 3; i++) {  
    digitalWrite(motorPins[i], LOW);  
  }  
}
```